

Experiment 5: Card Game Hand Sorting

Aim

To implement the **Optimized Bubble Sort** algorithm in Python to sort a player's cards after drawing a new card.

Problem Statement

A player has a nearly sorted hand of cards. After drawing one new card, use Bubble Sort to arrange the cards and count the number of passes required.

Algorithm

Step 1: Start.

Step 2: Read the number of cards.

Step 3: Read the card values.

Step 4: Apply Optimized Bubble Sort.

Step 5: Count the number of passes.

Step 6: Display the sorted cards and passes.

Step 7: Stop.

Source Code (Python)

main.py

```
1  # Card Game Hand Sorting
2
3  n = int(input("Enter number of cards: "))
4  cards = []
5
6  print("Enter card values:")
7  for i in range(n):
8      cards.append(int(input()))
9
10 passes = 0
11
12 for i in range(n - 1):
13     swapped = False
14     passes += 1
15
16     for j in range(n - i - 1):
17         if cards[j] > cards[j + 1]:
18             cards[j], cards[j + 1] = cards[j + 1], cards[j]
19             swapped = True
20
21     if not swapped:
22         break
23
24 print("Sorted Cards:", cards)
25 print("Number of Passes:", passes)
```

Output

```
Output
Enter number of cards: 7
Enter card values:
2
4
6
8
9
11
3
Sorted Cards: [2, 3, 4, 6, 8, 9, 11]
Number of Passes: 6

=== Code Execution Successful ===
```

Complexity Analysis

Aspect	Complexity
Nearly Sorted Hand	$O(n)$
Worst Case	$O(n^2)$
Space Complexity	$O(1)$

Result

The Optimized Bubble Sort algorithm successfully sorted the player's hand. Since the hand was nearly sorted, only a few passes were required, making Bubble Sort suitable for this scenario.